



引用&与&&

▼ 左值与右值

在说明引用前，首先要说明另外两个概念：左值与右值。

左值的英文简写为“lvalue”，右值的英文简写为“rvalue”。

很多人认为它们分别是“left value”、“right value”的缩写，其实不然。lvalue 是“locator value”的缩写，可意为存储在内存中、有明确存储地址（可寻址）的数据，而 rvalue 译为“read value”，指的是那些可以提供数据值的数据（不一定可以寻址，例如存储于寄存器中的数据）。

而这与赋值语法不谋而合，赋值左侧一定是有地址的变量，称为左值，而赋值右侧则可以是提供数据的字面量、变量、对象、表达式、函数的返回值等，称为右值。

但随着右值引用的使用，右值逐渐特指没有地址的数据，即字面量、表达式、函数的返回值等。

所有的变量，只要有名字，就是左值。

需要注意的是，涉及变量的表达式，如 `a*2`，也是右值，其本质是使用变量储存的值完成运算后返回的值。

通过左值进行初始化，会触发拷贝构造函数，系统会创建一个新的对象，并复制原对象的数据和资源。

通过右值进行初始化，会触发移动构造函数，系统将转移原对象的数据和资源。移动构造函数将原对象的资源指针“转交”给新对象，并把原对象的指针置为 `nullptr` 或一个空的状态，这样原对象不再拥有这些资源，从而避免了资源的重复释放。

`std::move` 本质就是将左值引用改变为右值引用，这样通过 `std::move` 进行初始化的变量将会通过右值进行初始化，触发移动构造函数。

▼ 左值引用：&

非常量引用

普通的引用，正常情况下只能操作 C++ 中的左值，无法对右值添加引用

```
C++  
  
int num = 10;  
const int num2 = 100;  
int& b = num; //正确  
int& c = 10; //错误  
float& d = num; //错误  
int& e = num2; //错误
```

普通的非常量引用，必须引用同类型的变量。

修改引用与被引用的两个变量其中一个时，两个变量的值会一起改变。

非常量引用不可引用常量变量。

常量引用

不支持为右值建立非常量左值引用，但允许使用常量左值引用操作右值。也就是说，常量左值引用既可以操作左值，也可以操作右值。

```
C++  
  
int num = 10;  
const int num2 = 100;  
const int& b = num; //正确  
const int& c = 10; //正确  
const float& d = num; //正确  
const int& e = num2; //正确
```

使用常量引用可以引用右值，但是同时因为“const”的限制，引用右值的变量本身无法修改，而且因为引用的右值没有地址可寻，也无法通过被引用的右值修改，所以是没有办法可以修改的。

当使用常量引用左值时，如果类型完全一致，则作用与非常量引用类似，虽然本身不能修改，但是可以通过被引用的变量修改；如果类型不一致，则会被引用的变量中储存的值作为右值引用，因为本质是引用的变量中存储的右值而不是变量，所以无法通过被引用变量修改。

▼ 右值引用：&&

为了弥补非常量左值引用只能引用左值，常量左值引用在引用右值后无法修改的缺憾，所以引入了右值引用这一方式。右值引用特指非常量右值引用，原因后面会说。

```
C++  
  
int num = 10;  
const int num2 = 100;  
int&& a = num; //错误，右值引用不能初始化为左值  
int&& a = 10; //成功，但是其实已经退化为了左值  
const int&& b = num; //错误，右值引用不支持左值  
const int&& c = 100; //成功，但是其实已经退化为了左值  
const int&& d = num2; //错误，常量变量有具体地址，也是左值
```

与非常量左值引用一样只能引用左值一样，右值引用只能引用右值，而且可以修改右值。

但常量右值引用与常量左值引用既可引用左值也可引用右值不同，常量右值引用依然只能引用右值，不能引用左值，这点不如常量左值引用，而且因为“const”的限制，引用的右值无法修改，这点不如非

常量右值引用，所以常量右值引用并没有实际用处，因此右值引用特指非常量右值引用。

右值引用 `T&&` 确实是一种类型，但是只要右值引用拥有了变量名，那么它实际上已经退化为了左值，此时可以称它为**类型为右值引用的左值**。

所以本质上右值引用是无法稳定存在的，字面量是右值本身而不是右值引用，右值引用类型的变量本质是左值。

对于明确指定右值引用类型的函数参数，必须接受右值或 `std::move` 转换的右值引用，而不能接受已经退化为左值的右值引用变量。

传参之后，虽然参数类型为右值引用，但是函数参数需要有名称，所以函数内该变量实际上已经退化为了左值（并非左值引用）。此时若还想将其当作右值使用，需要使用 `std::forward` 转发或使用 `std::move` 强制转换为右值。

▼ 引用折叠

在 C++ 中是不能直接在代码里写“引用的引用”的（比如 `int& && a` 会报错）。

但当模板推导或 `typedef` 发生时，系统内部会不可避免地出现这种情况。为了处理这些嵌套引用，编译器制定了一套合并规则。

在 C++ 模板中，引用的引用会发生折叠。

原始引用	叠加的引用	折叠结果
<code>&</code> （左值）	<code>&</code> （左值）	<code>&</code>
<code>&</code> （左值）	<code>&&</code> （右值）	<code>&</code>
<code>&&</code> （右值）	<code>&</code> （左值）	<code>&</code>
<code>&&</code> （右值）	<code>&&</code> （右值）	<code>&&</code>

要参与合并的引用中有一个是**左值引用（&）**，结果就一定是左值引用。只有当两者全都是**右值引用（&&）**时，结果才是右值引用。

▼ 万能引用

在一些编译器自动推导类型处，如果使用 `auto&&` 或 `T&&`，就可以实现万能引用。

万能引用通常出现在函数模板的参数中，因为函数模板是自动推导最多的场合。

在模板类中，**模板参数并不会自动进行引用类型推导**，因为模板类实例化时必须显式声明，在实例化后，模板类中依赖该模板参数的成员属于已经确定的类型，而不是自动推导的。

万能引用也可以用来接收不确定是左值还是右值的返回值，因为返回值一般也可以自动推导。

需要注意的是，右值一般被推导为非引用类型，因为右值传参一般为移动构造，不过由于万能引用本身之后的 `&&`，非引用类型可以变为右值引用。

万能引用基于引用折叠的机制，可以自动根据传递的是左值还是右值进行类型推导。

在传递左值或左值引用时，`T` 会被推导为某种具体类型的 `&`，此时根据引用折叠，`& &&` 会被还原为 `&`。

在传递右值时，编译器会将 `T` 推导为某种非引用类型，此时无需进行引用折叠，万能引用的 `&&` 直接表示右值引用。

在传递右值引用类型变量时，本质是传递左值，所以也会被推导为 `&`，因为 `forward` 会按照推导得到的类型转发，所以此时即使使用 `forward` 转发，也是转发为左值引用。。

传入实参的类型/属性	表达式属性	T 的推导类型	arg 的最终类型
左值(<code>i</code>)	左值	<code>int&</code>	<code>int&</code>
右值(<code>10</code>)	右值	<code>int</code>	<code>int&&</code>
左值引用变量(<code>int& lref</code>)	左值	<code>int&</code>	<code>int&</code>
右值引用变量(<code>int&& rref</code>)	左值	<code>int&</code>	<code>int&</code>

▼ 数组引用

数组在值传递时，除了直接传递指针，还有一种语法为 `void f(int arr[]);`，这种语法更直观表示 `arr` 为一个数组地址，而不是普通的指针。

在这种语法的基础上，可以在形参上显式标明数组大小，如 `void f(int arr[3]);`，但其实在实际传递时会发生退化，直接只传递了数组的指针，数据的大小基本等同于注释。

但是数组引用在语法层面必须有大小信息，如 `void f((&arr)[3]);`，编译器必须知道数组的具体大小，否则无法通过编译。

为了避免固定数组的大小，可以使用常量模板推导数组大小，但是数组大小本身是不可省略的：

C++

```
template <size_t N>
void f(int (&arr)[N]) {
    // N 就是数组大小
}
```